

Lecture 6: Django Apps, URLs, and Views

Backend Python + Django | Second Year – Faculty of Computers & AI

Apps • URLs • Views • Routing •
Render

Dr. Mohamed Nour Eldien

Where are we now?

Lecture 5 recap + today's position

- In Lecture 5 we created a Django project and ran the development server.
- Today we move from “project exists” to “project can serve real pages”.
- We will create an app, register it, build URLs, write views, and test the result.
- This is the bridge from setup to real web application behavior.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

Learning Objectives

By the end of this lecture, students should be able to:

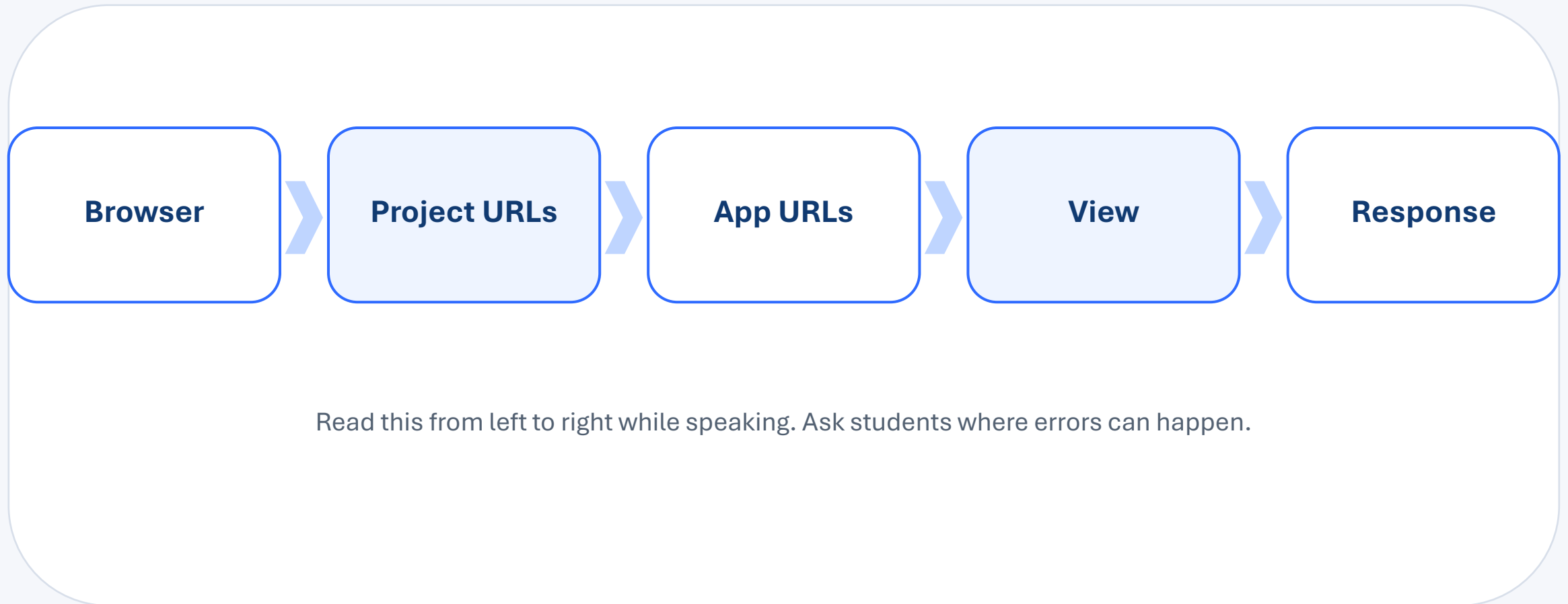
- Explain the difference between a Django project and a Django app.
- Create a new app and register it in `INSTALLED_APPS`.
- Define URL patterns at both project level and app level.
- Write function-based views and return `HttpResponse` or rendered templates.
- Pass basic data from a view to the browser.
- Debug common routing and import errors.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Big Picture of Today

One request, many Django pieces



What is a Django App?

The basic reusable unit inside a Django project

- A Django app is a self-contained module that handles one part of the site.
- An app usually has its own views, URLs, models, templates, and admin settings.
- Examples: students app, courses app, accounts app, library app.
- Large systems are easier to build when functionality is split into multiple apps.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

Project vs App

Think of the project as the whole site and the app as a department

Project

- Global settings
- Main URLs file
- Server configuration
- Contains many apps

App

- Specific feature or domain
- Own views and logic
- Can have its own URLs and templates
- Can be reused in another project

Real-Life Analogy

University campus analogy

- Project = the whole university campus.
- App = one building or one department inside that campus.
- URLs = signs that guide visitors to a destination.
- Views = the employee or office that handles the request.
- Response = the final paper, result, or screen shown to the student.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Creating a New App

Command line step

```
1 python manage.py startapp students
```


Files Created by startapp

What Django gives us automatically

- `admin.py`: register models in admin panel later.
- `apps.py`: app configuration class.
- `models.py`: define database structure later.
- `tests.py`: place automated tests.
- `views.py`: write request handling logic.
- `migrations/`: database migration history.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Why Many Apps?

Separation of concerns

- Each app owns a specific responsibility.
- Code becomes easier to read and maintain.
- Teams can work on different parts in parallel.
- Bugs become easier to isolate.
- Some apps can be reused in future projects.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Register the App in settings.py

Without registration Django does not know your app

```
1 INSTALLED_APPS = [  
2     'django.contrib.admin',  
3     'django.contrib.auth',  
4     'django.contrib.contenttypes',  
5     'django.contrib.sessions',  
6     'django.contrib.messages',  
7     'django.contrib.staticfiles',  
8     'students',  
9 ]
```

Common Mistake: App Not Registered

Symptoms students often see

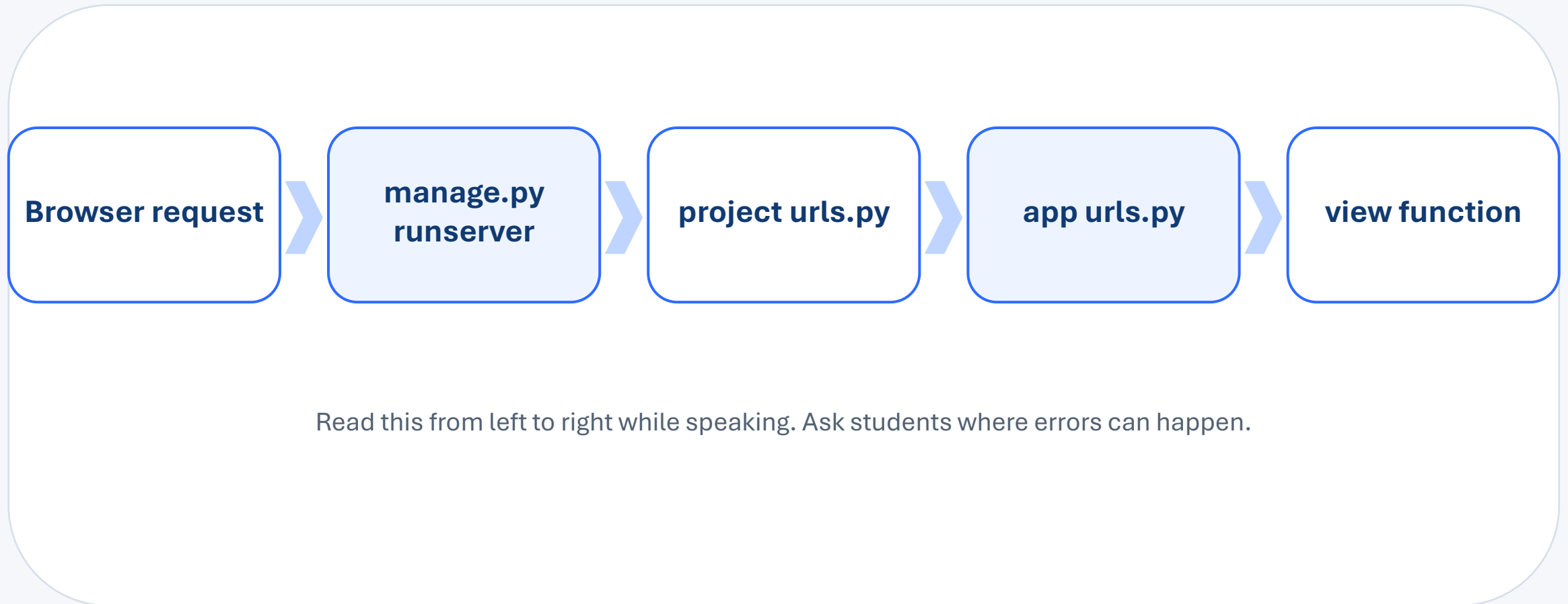
- Templates not discovered as expected later.
- Models and admin registration not working correctly later.
- Confusion because the folder exists but behavior is incomplete.
- Always check settings.py after creating any new app.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

How a Request Finds Its View

Routing pipeline



What is URL Dispatcher?

The component that maps a path to a view

- Django uses URL patterns to decide what code should run.
- A URL pattern has two main parts: the path and the view.
- Sometimes we also give the URL a name for later use.
- If no pattern matches, Django returns 404 Not Found.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Project-Level URLs

Main urls.py in the project folder

```
1 from django.contrib import admin
2 from django.urls import path, include
3
4 urlpatterns = [
5     path('admin/', admin.site.urls),
6     path('students/', include('students.urls')),
7 ]
```

Why include() Matters

Clean routing architecture

- Project urls stay short and readable.
- Each app manages its own internal paths.
- New apps can be plugged in without cluttering the main file.
- Routing becomes easier to debug and maintain.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

App-Level URLs

students/urls.py

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('', views.home),
6     path('about/', views.about),
7 ]
```

Your First View Function

A view can return simple text

```
1 from django.http import HttpResponse
2
3 def home(request):
4     return HttpResponse('Welcome to the Students App')
```

Second View Example

Another page in the same app

```
1 from django.http import HttpResponse
2
3 def about(request):
4     return HttpResponse('This app handles student pages')
```

What Does request Mean?

Every view receives the request object

- The browser sends a request to the server.
- Django packages that request into a Python object.
- The view can read method, headers, query data, form data, and more.
- Even when we do not use it yet, request must exist in the function signature.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

View = Business Logic Entry Point

Where decisions are made

- Check the request.
- Prepare or fetch data.
- Choose what response to return.
- Later, views may connect templates, forms, models, and authentication.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Returning HTML Inside HttpResponse

Possible, but not the best long-term approach

```
1 from django.http import HttpResponse
2
3 def home(request):
4     return HttpResponse('<h1>Students Home</h1><p>Welcome!</p>')
```

Using render() Instead

Better separation between logic and HTML

```
1 from django.shortcuts import render
2
3 def home(request):
4     return render(request, 'students/home.html')
```

Why Templates Are Better

Preparing for next lecture

- HTML stays readable.
- Views stay focused on logic.
- Designers and backend developers can work separately.
- Templates can later receive dynamic data from views.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Passing Context Data to a Template

View sends data, template displays it

```
1 from django.shortcuts import render
2
3 def home(request):
4     data = {
5         'title': 'Students Home',
6         'message': 'Welcome to the students portal'
7     }
8     return render(request, 'students/home.html', data)
```

Dynamic URL Parameters

Capture values from the path

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('student/<int:id>/', views.student_details),
6 ]
```

View with a URL Parameter

The parameter arrives as a function argument

```
1 from django.http import HttpResponse
2
3 def student_details(request, id):
4     return HttpResponse(f'Student ID = {id}')
```

Common Path Converters

Django gives us converters out of the box

Useful converters

- str: text without slash
- int: integer value
- slug: letters, numbers, hyphen, underscore
- uuid: UUID value

Example paths

- <str:name>/
- <int:id>/
- <slug:code>/
- <uuid:item_id>/

Practice 1: Predict the Final URL

In-class activity

Work in pairs. Read, discuss, then code.

- If project urls contain `path('students/', include('students.urls'))`, what full path opens `path('about/', views.about)` inside the app?
- If the app has `path("", views.home)`, what full browser path reaches that view?
- What full path reaches `path('student/<int:id>', views.student_details)` when `id = 8`?

Teaching hint

Give students 4–5 minutes first. Then ask one team to explain their routing or view code on the board.

Order of URL Patterns Matters

First match wins

- Django checks patterns from top to bottom.
- Place more specific patterns before broad catch-all patterns.
- Bad ordering can send the request to the wrong view or hide another route.
- This becomes especially important in bigger apps.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Function-Based Views (FBV)

Our focus in this course stage

- A function-based view is a normal Python function.
- Easy to read for beginners.
- Great for understanding request-response flow clearly.
- Later Django also supports class-based views, but we delay them now.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Three Simple Views in One App

A realistic starter example

```
1 from django.http import HttpResponse
2
3 def home(request):
4     return HttpResponse('Students Home Page')
5
6 def contact(request):
7     return HttpResponse('Contact the students office')
8
9 def help_page(request):
10    return HttpResponse('Help page')
```


Map Those Views to URLs

Complete the routing

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('', views.home),
6     path('contact/', views.contact),
7     path('help/', views.help_page),
8 ]
```

What Makes a Good View?

Simple rules for beginners

- Clear name that reflects page purpose.
- Short and readable code.
- Return one obvious response.
- Avoid mixing too many unrelated tasks in one view.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

JSON Response Preview

Views can return machine-readable data too

```
1 from django.http import JsonResponse
2
3 def api_status(request):
4     return JsonResponse({'status': 'ok', 'app': 'students'})
```

Naming URL Patterns

A small habit that helps later

- Named URLs make templates and redirects cleaner.
- Example: `path("about/", views.about, name="about")`
- Names reduce hard-coded links later.
- Names become more useful as projects grow.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Named URL Example

One line, better maintainability

```
1 urlpatterns = [  
2     path('', views.home, name='home'),  
3     path('about/', views.about, name='about'),  
4 ]
```

Where Do Templates Live?

Preview before the dedicated lecture

- Common app-based pattern:
app_name/templates/app_name/file.html
- Example: students/templates/students/home.html
- This naming avoids conflicts between apps.
- APP_DIRS=True helps Django find templates inside apps.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

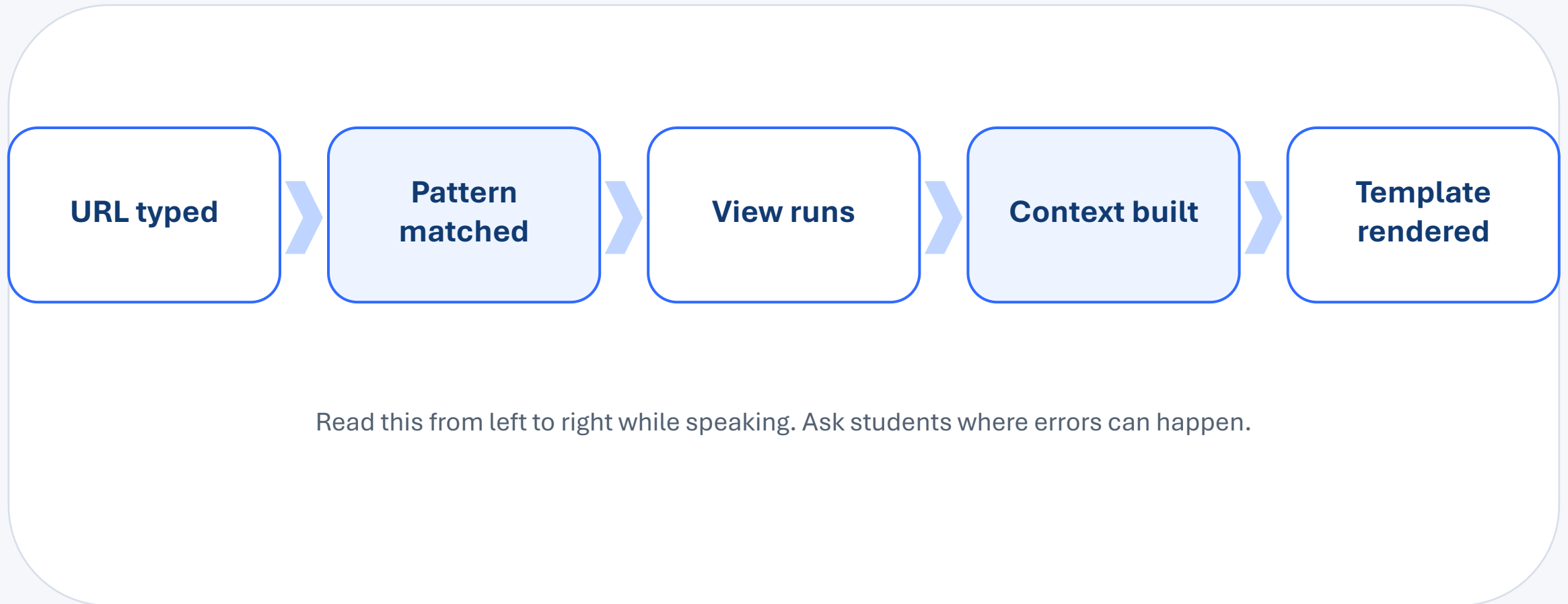
Minimal Template Example

home.html

```
1 <h1>{{ title }}</h1>  
2 <p>{{ message }}</p>
```

End-to-End Example

From URL to HTML on screen



Debugging Tip 1: 404 Not Found

Most routing problems show up here

- Check the full path typed in the browser.
- Check whether project urls include the app urls.
- Check whether app urls contain the specific pattern.
- Check trailing slash consistency.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

Debugging Tip 2: Import Errors

When Django cannot find your view or module

- Check file name and function name carefully.
- Ensure from . import views is correct in app urls.
- Ensure the function exists in views.py.
- Read the traceback slowly instead of panicking.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

A Small but Complete Working Example

students/views.py + students/urls.py

```
1 # views.py
2 from django.http import HttpResponse
3
4 def home(request):
5     return HttpResponse('Home page')
6
7 def about(request):
8     return HttpResponse('About page')
9
10 # urls.py
11 from django.urls import path
12 from . import views
13
14 urlpatterns = [
15     path('', views.home),
16     path('about/', views.about),
17 ]
```

Practice 2: Build a Courses App Mentally

In-class activity

Work in pairs. Read, discuss, then code.

- What command creates an app called courses?
- What line must be added to `INSTALLED_APPS`?
- What line must be added to `project urls.py`?
- What two views would you create first in the app?
- What browser paths should open these views?

Teaching hint

Give students 4–5 minutes first. Then ask one team to explain their routing or view code on the board.

path() Function Parameters

Understand each piece

- First argument: route string.
- Second argument: view function.
- Third optional argument: name.
- Put together, they define one URL rule.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Example with name=

Complete pattern syntax

```
1 path('contact/', views.contact, name='contact')
```

Trailing Slash in Django

A detail students must notice

- Many Django examples end paths with /.
- Keep your style consistent across the project.
- Typing the wrong slash pattern may lead to confusion during testing.
- Be careful when comparing the route string and the browser URL.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Good App Names

Readable names help teams

- Use meaningful singular or plural names such as students, courses, library.
- Avoid vague names like app1, data, temp, or test123.
- Consistency across folder names, app labels, and URLs helps debugging.
- Choose names that describe responsibility clearly.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

Function-Based View vs HTML File

What stays where?

Inside the view

- Logic
- Read request
- Prepare data
- Choose response type

Inside the template

- HTML structure
- Displayed text and layout
- Presentation of context variables
- Visual page content

One View, Different Responses

Views can choose how to respond

```
1 from django.http import HttpResponse, JsonResponse
2
3 def demo(request):
4     if request.GET.get('format') == 'json':
5         return JsonResponse({'page': 'demo'})
6     return HttpResponse('Demo page')
```

Mini Lab Workflow

How to practice in the lab efficiently

- Create app → register app → create app urls → include app urls → write views → test in browser.
- After every small step, save files and test quickly.
- Do not write ten steps then test once.
- Use the terminal output to confirm server behavior.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

Practice 3: Fix the Broken Routing

In-class activity

Work in pairs. Read, discuss, then code.

- A student wrote `include('student.urls')` but the app folder is `students`.
- Another student forgot to create `students/urls.py`.
- Another student wrote `path('about', views.about)` and typed `/students/about/` in the browser.
- Explain the fix for each case.

Teaching hint

Give students 4–5 minutes first. Then ask one team to explain their routing or view code on the board.

Common Error Messages You May See

Read them as clues

- `ModuleNotFoundError`
- `ImportError`
- Page not found (404)
- `ViewDoesNotExist` or similar traceback references

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Debug Example: Missing include import

Small mistake, easy fix

```
1 # Wrong
2 from django.urls import path
3
4 # Correct
5 from django.urls import path, include
```

Debug Example: Wrong View Reference

Another common beginner issue

```
1 # Wrong
2 path('about/', view.about)
3
4 # Correct
5 path('about/', views.about)
```

Why We Test in the Browser

Immediate feedback matters

- The browser is the easiest place to verify routing quickly.
- If the page loads, routing and view are likely connected correctly.
- If not, the URL or traceback usually points to the problem.
- Test one page at a time.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

How This Connects to Backend Thinking

Not just Django syntax

- Client asks for a resource.
- Server routes the request.
- Backend logic decides the response.
- Django is implementing the same general backend idea using Python.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

Quick Quiz 1

Check understanding now

Which file usually contains the line `include("students.urls")`?

A. `students/views.py`

B. `project urls.py`

C. `students/models.py`

D. `settings.py`

Answer

B. `project urls.py`

Quick Quiz 2

Concept check

A view function must receive which required first argument?

A. response

B. request

C. template

D. id only

Answer
B. request

Quick Quiz 3

Routing check

If project urls route `students/` to `students.urls`, and app urls contain `path("about/", views.about)`, what is the full path?

A. `/about/`

B. `/students/about/`

C. `/students/`

D. `/about/students/`

Answer

B.

`/students/about/`

Best Practices Summary

Small rules with big impact

- Use meaningful app names.
- Register every app immediately after creating it.
- Keep urls organized using include().
- Write small, clear views.
- Test every route in the browser.
- Read tracebacks carefully.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

What We Are Not Covering Yet

To keep the foundation clear

- Class-based views
- Forms and POST handling in depth
- Database queries with models
- Authentication and permissions
- APIs with Django REST Framework

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

Connection to Lecture 7

Why templates are the next natural step

- Today we saw `render(request, template)`.
- Next lecture explains templates in detail.
- We will build dynamic HTML pages instead of plain text responses.
- That means your Django site will start to look like a real website.

Example to mention

Use a real page such as `/students/about/` or the university portal home page to connect concept to reality.

Homework

Complete these tasks before the next class

- Create a new Django app called courses.
- Register it in INSTALLED_APPS.
- Connect it to project urls using include().
- Create at least three views: home, about, and contact.
- Create matching URL patterns and test them in the browser.
- Write down any errors you faced and how you fixed them.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

Speaking Prompt for the Lab

How to explain your solution aloud

- What app did you create and why?
- How did you register the app?
- How did the request move from project urls to app urls?
- What does each view return?
- Which problem did you face during testing?

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

Final Summary Map

One sentence per concept

- App = a focused module inside the project.
- INSTALLED_APPS = activates the app inside Django.
- Project urls = main entry for routing.
- App urls = routes inside one app.
- View = Python function that handles the request and returns a response.
- Render = return an HTML template, usually with data.

Example to mention

Use a real page such as /students/about/ or the university portal home page to connect concept to reality.

Thank You

Next time: Templates and Dynamic Pages

Homework first. Templates next.